SmartBus Phase IV

AI/ML & Semantic Ontology Integration

CSE 598 — Software Integration and Engineering

Sherzod Omonov | Spring 2026

Executive Summary

Hybrid ML

Recommendation System
CF + Content-Based

10 Classes

OWL 2 Ontology
17 Relationships, 385 Triples

< 30 ms

End-to-End Latency
AI + Semantic Pipeline

Phase IV transforms SmartBus from a data-integration platform into an intelligent, knowledge-aware booking system. A trained ML model recommends optimal routes per user, an OWL 2 ontology formally represents domain knowledge, and a bidirectional integration layer connects learning with reasoning — closing the AI-semantic loop.

Phase IV Objectives

AI/ML Route Recommendations

Train a hybrid collaborative + content-based model to suggest optimal routes based on each user's booking history and travel profile.

Semantic Ontology (OWL 2)

Design and implement a 10-class, 17-relationship ontology in Turtle format representing the full SmartBus domain with inference rules.

AI-Ontology Bidirectional Integration

Feed ontology features into ML scoring, store predictions as RDF triples, and combine ML + semantic reasoning for enriched recommendations.

Full-Stack Integration

Expose prediction and semantic endpoints via Spring Boot gateway, display recommendations in the Buy Ticket UI with confidence scores.

Problem Statement & Dataset

The Problem

Given a passenger's booking history, recommend the 3 most relevant routes they have not yet booked.

The model must learn preferences from historical patterns rather than applying hardcoded rules.

ML Type: Recommendation System (implicit feedback)

Framework: Python 3.14 + Scikit-learn

Evaluation: Per-user temporal hold-out (75/25 split)

Dataset at a Glance

300	Registered Users
10	Bus Routes
5,174	Total Bookings
3,858	Completed (74.6%)
1,507	(user, route) Pairs
50.2%	Matrix Density

Hybrid Recommendation Algorithm

```
score(user, route) = α × CF_score  +  (1 − α) × CB_score      α = 0.3, k = 30
```

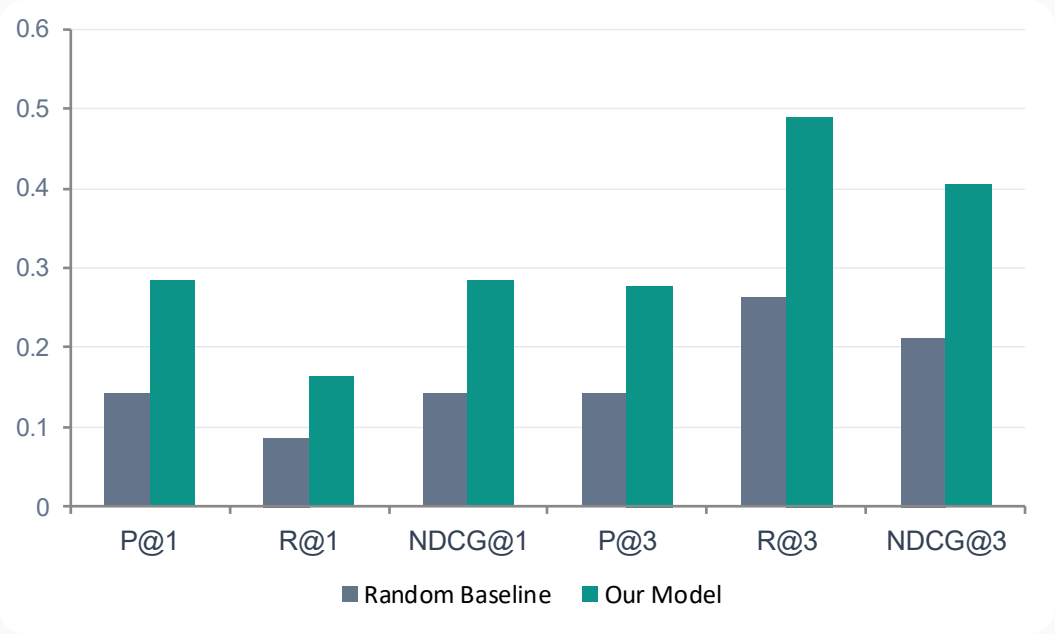
Collaborative Filtering (CF)

- User-user cosine similarity on the implicit-rating interaction matrix
- Select top-30 most similar users as neighbours
- Weighted average of neighbours' ratings per route
- Captures "users like you also book this route" signal

Content-Based Filtering (CB)

- 21-dimension route feature vector (price, origin, destination, hour, tier)
- User preference profile = weighted avg of booked route features
- Cosine similarity between user profile and candidate routes
- Handles new routes with no interaction history

Model Performance & Results



Key Highlights

+91.6%

NDCG@3 lift over random

49%

Recall@3 — nearly half of test bookings recovered

100%

Coverage — all 10 routes recommended

>84%

of theoretical P@3 ceiling reached

Hyperparameter Tuning

k \ α	0.3	0.4	0.5	0.6	0.7	0.8
5	0.4023	0.4011	0.3940	0.3963	0.3948	0.3808
10	0.3990	0.3984	0.3876	0.3887	0.3960	0.3818
20	0.4050	0.3984	0.3980	0.3931	0.3945	0.3832
30	**0.4051**	0.3971	0.3902	0.3845	0.3846	0.3683
50	0.3990	0.3970	0.3967	0.4000	0.3905	0.3927

NDCG@3 scores across 30 grid-search combinations | Best: k=30, α=0.3 (highlighted in teal)

Lower α (0.3) means content-based filtering dominates. With 10 routes and strong feature coverage, CB is highly informative. k=30 neighbours reduces noise without diluting signal. NDCG@3 range: 0.3683–0.4051 — stable learning across all configurations.

Ontology Model — OWL 2 / Turtle

10 Classes (Entities)

- User — registered passenger
- Route — named bus route
- Stop — bus terminal / stop
- Bus — physical vehicle
- Booking — ticket reservation
- Payment — transaction record
- Schedule — time-bound route run
- Recommendation — AI suggestion
- PriceTier — fare category (L/M/H)
- RouteZone — geographic zone

17 Object Properties

- hasBooking / bookedBy (inverse)
- departsFrom / arrivesAt
- onRoute, paidWith, operatedBy
- frequentlyTravels (inferred)
- interactedWith (property chain)
- inSameZoneAs (symmetric, inferred)

385	284	3
Ontology Triples	Sample Data Triples	Route Zones

Ontology Inference Rules

R1

Frequent Route Preference

If a user has ≥3 completed bookings on a route, infer frequentlyTravels relationship.

R2

Origin Stop Preference

User's most common departure stop becomes their prefersOrigin property.

R3

Zone Proximity (symmetric)

If two stops share a zone, they are inSameZoneAs each other.

R4

Route Candidate

If a route departs from the user's preferred origin, it is a candidate recommendation.

R5

High-Confidence Rec

If user frequently travels route AND prefers its price tier, confidence = HIGH.

```
OWL 2 Property Chain:  interactedWith  ≡  hasBooking ∘ onRoute
```

AI-Ontology Integration Architecture

AI → Ontology

- ML predictions stored as RDF triples
- Confidence scores become semantic properties
- Knowledge graph enriched with predicted_to_book relationships
- Enables explainability via SPARQL queries

Ontology → AI

- Semantic features boost ML scores (origin preference, zone, tier)
- Inference rules (R1–R5) add semantic_boost to enriched_score
- Ontology constraints validate and filter predictions
- Domain reasoning improves recommendation quality

Combined Decision Example

User 157 | Frequent traveler | Home stop: Downtown Terminal | Preferred tier: MEDIUM

Rank	Route	From → To	ML Score	Semantic Boost	Final Score	Hit?
1	**SB-101**	Downtown → Airport	0.375	+R2, R4	**0.425**	—
2	**SB-201**	Airport → City Center	0.262	+R1	**0.312**	✓
3	**SB-103**	Downtown → Harbor	0.202	+R2, R4	**0.252**	✓

Test routes: {SB-103, SB-201} — 2/2 hits (perfect recall@3)

Semantic boost from rules R2 (origin preference) and R4 (route candidate) elevated SB-101 and SB-103 which depart from the user's preferred Downtown Terminal stop. Rule R1 boosted SB-201 based on frequent travel history.

System Demonstration

1. **User opens Buy Ticket page**
   Frontend calls /api/v1/frontend/recommendations

2. **Gateway proxies to Flask ML server**
   JWT validated, request forwarded to Python inference

3. **ML model predicts top-N routes**
   Hybrid CF+CB scoring in 2–5 ms

4. **Semantic enrichment applied**
   Ontology features boost scores, triples stored in KG

5. **Enriched recommendations returned**
   Cards with route, score, confidence displayed in UI

API Endpoints

GET `/recommendations`
ML-only route recommendations

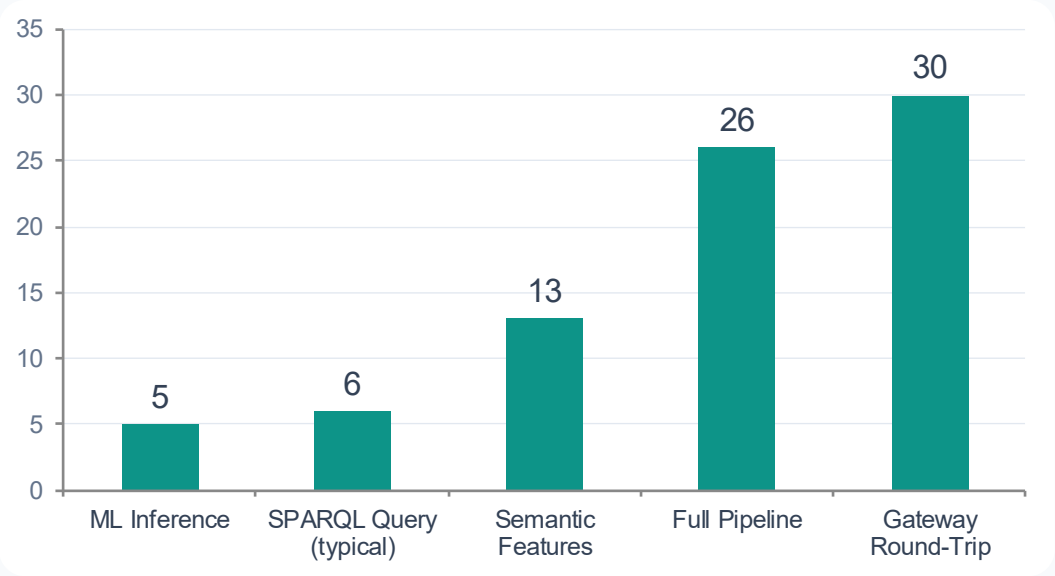
GET `/intelligent/recommend`
AI + Ontology enriched recs

GET `/intelligent/explain`
Per-route reasoning breakdown

GET `/semantic/find-related`
Entity relationship traversal

GET `/semantic/query`
Ad-hoc SPARQL queries

Performance Benchmarks



SLA Compliance

- SPARQL < 1000 ms
  < 25 ms
- ML Inference < 100 ms
  < 8 ms
- Full Pipeline < 1000 ms
  < 50 ms
- All Tests Pass
  95/95

KG Startup: 136 ms one-time | Model Cold Load: ~400 ms one-time | 675 total triples in memory

Technology Stack

Backend	ML / AI	Ontology	Frontend	Infrastructure
Java 25, Spring Boot 4.0	Python 3.14	OWL 2 / Turtle format	HTML / CSS / JS	Docker Compose
Maven multi-module	Scikit-learn (cosine sim)	RDFLib 7.0 (in-memory)	Webpack build	Kafka 3.9 messaging
Spring Security (JWT)	Flask REST server	SPARQL query engine	Bootstrap styling	PostgreSQL per service
PostgreSQL 17	Joblib model persistence	5 inference rules	Fetch API integration	Gunicorn (production)

Challenges & Lessons Learned

Challenge	Lesson / Solution
Small catalogue (10 routes) limits P@K ceiling	Focus on relative metrics (NDCG lift over baseline) rather than absolute precision
Synthetic data may not reflect real booking patterns	Built pipeline to swap generate_data.py with live PostgreSQL export seamlessly
Bridging Java backend with Python ML model	Flask REST microservice provides clean HTTP boundary; gateway proxies transparently
Keeping ontology consistent with evolving schema	OntologyValidator runs domain/range/functional checks on every KG load
Cold-start users with no booking history	Popularity fallback ensures every user gets meaningful recommendations

Future Improvements

Live Data Integration

Replace synthetic CSV with real-time PostgreSQL export; retrain model on actual booking patterns.

Advanced ML Algorithms

Implement matrix factorization (SVD) and temporal decay weighting for better generalization.

Dedicated Triple Store

Migrate from in-memory RDFLib to Apache Jena TDB2 or Blazegraph for production scale (>50K triples).

Explicit User Feedback

Collect thumbs-up/thumbs-down on recommendations to create a closed feedback loop.

Real-Time Model Updates

Implement online learning or periodic retraining triggered by new booking volume thresholds.

Thank You

Questions & Discussion

Hybrid ML	10-Class	< 30 ms	95/95
Recommendation Engine	OWL 2 Ontology	Full Pipeline Latency	Tests Passing

Sherzod Omonov | sherzodbeek98@gmail.com | CSE 598 Spring 2026